Self-Taught Semi-Self Speculative Decoding (S4D): A Learned Routing Policy for Hierarchical Verification

viasd-rl experiments

June 20, 2026

Abstract

We reimplement VIA-SD (arXiv:2606.12243) — speculative decoding with a slim, layer-skipped intermediate verifier q' — and replace its hand-tuned (α_1, α_2) confidence thresholds with a **learned per-token gating policy**. We study three RL formulations (REINFORCE, GRPO, and a per-token-reward variant) and their reward/credit-assignment design. On a GSM8K slice with a Qwen2.5 0.5B→14B pair, the learned gate **dramatically outperforms the paper's fixed thresholds** (accuracy 0.21–0.40 → 0.88–0.91) at fewer expensive verifier calls. We also report honest negatives: the slim-verifier middle tier only pays off when the gate is learned, and scaling the verifier alone (without the drafter) hurts an accept-heavy policy.

1 Introduction

Why speeding up speculative decoding matters. Autoregressive LLM inference is *memory-bandwidth bound*: generating each token streams all model weights from HBM, so latency is dominated by a long chain of *sequential* large-model forward passes. For a 14B model this is the bottleneck behind both interactive latency and serving cost. *Speculative decoding* (SD) attacks it by letting a cheap *drafter* propose several tokens that the large *verifier* checks in *parallel* (one forward), amortizing one expensive pass over multiple accepted tokens; it is lossless and gives a 1.3–2× wall-clock speedup in practice. The single lever is the *acceptance rate* — every rejected token forces a full-model step — so reducing rejections directly buys speed.

VIA-SD, and where it can be improved. VIA-SD inserts a *slim intermediate verifier* q' (a layer-skipped copy of the target q) so that "middle-zone" tokens — where the drafter is plausible but not certain — are resolved cheaply by q' instead of escalating to the full model. This three-tier hierarchy (accept / regenerate-with-q' / escalate-to-q) cuts full-model invocations. *But* the routing among tiers is decided by two **hand-tuned global thresholds** (α_1, α_2) on q''s confidence: a brittle, context-blind rule that must be re-tuned per model and task. That is precisely the opening — choosing a verification tier per token is a sequential, context-dependent control problem, exactly what a *learned* policy should do better than fixed thresholds.

Our contribution. We replace the fixed thresholds with a **learned per-token gating policy**: a tiny MLP that, from q-free features, routes each drafted token to accept/regenerate/escalate (Fig. 1). We train it by imitation then reinforcement learning, and study three RL formulations (REINFORCE, GRPO, a per-token reward) together with their reward and credit-assignment design. On GSM8K with a Qwen2.5 0.5B→14B pair the learned gate beats the fixed-threshold rule by a wide margin (accuracy 0.21–0.40 → 0.88–0.91) at fewer expensive verifier calls — alongside honest negatives about where the slim tier does and does not pay off.

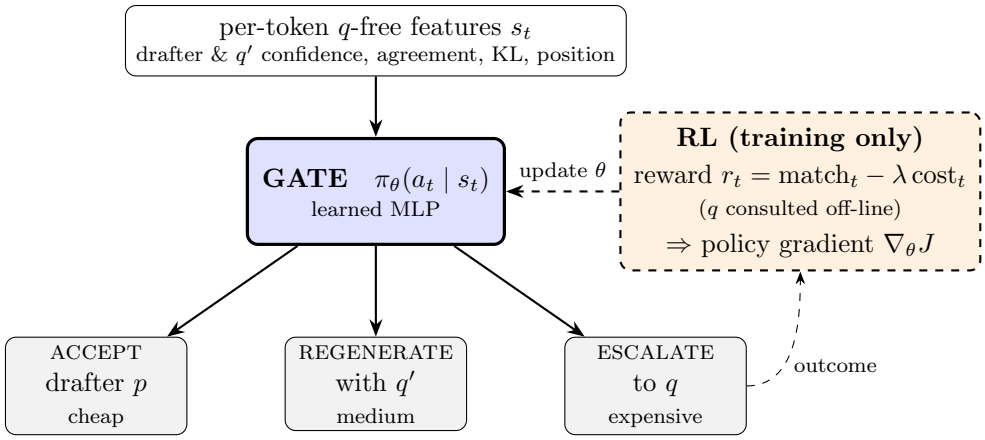

Figure 1: Method overview — the learned **gate** is the core. For each drafted token it reads q-free features s_t and routes to ACCEPT (drafter), REGENERATE (slim q'), or ESCALATE (full q). **RL slots in only at training time** (dashed): each action's outcome yields a reward $r_t = \text{match}_t - \lambda\,\text{cost}_t$ (with q consulted off-line), and a policy-gradient step updates the gate parameters θ. At inference the dashed path is removed and the gate runs from q-free features alone.

2 Background: VIA-SD and DIMR

Speculative decoding (SD). A cheap *drafter* p autoregressively proposes a block of γ candidate tokens; the expensive *target* verifier q scores all γ in a single parallel forward and accepts the longest prefix consistent with its own distribution, recomputing the first rejected token. This is *lossless* (output equals sampling/greedy from q) and trades extra compute for fewer *sequential* large-model steps. Its single lever is the *acceptance rate*: each rejection truncates the block and forces a full-model step.

VIA-SD's contribution. The paper observes that many rejected tokens lie in a "middle zone" — the drafter is close but not fully reliable — and do not actually need the full model. It inserts a *slim intermediate verifier* q' between p and q, turning the binary accept/reject into a three-way *hierarchical* decision per token: **accept** p's token when q' is highly confident; **regenerate** with q' for medium-confidence (middle-zone) tokens; **escalate** to the full q only for low-confidence tokens. By resolving middle-zone tokens at q', it reduces how often the expensive q is invoked relative to standard two-tier SD. The routing is governed by two fixed confidence thresholds (α_1, α_2) on q''s confidence ratio (reported best at α_1=0.5, α_2=0.3); the scheme is mildly *lossy*, with a tolerance α controlling how far accepted tokens may deviate. The paper motivates the intermediate stage information-theoretically: in KL geometry an intermediate distribution can lower the *cumulative* divergence on the path $p \to q' \to q$ below the direct $p \to q$, whereas total-variation (which exactly equals the lossless rejection rate) forbids this.

DIMR (Dynamic Intra-Model Routing): how q' is built. Rather than a separate model, q' is a *routed sub-model of q itself*: a binary mask $z \in \{0,1\}^L$ selects which of q's L decoder layers to keep, $q' = \Pi_z(q)$, skipping the rest (each decoder block is residual, so dropping it is an identity on the residual stream — dimensionally safe). q' *shares* q's embeddings and output head, so it adds no parameters and only changes inference dynamics. DIMR is the *offline search* for the mask z^*: it minimizes an accumulated KL-style cost between q and q'_z over a calibration window (the paper's Eq. 12), combining random search with Bayesian/greedy refinement, keeping first/last layers and skipping ∼45% of the middle. The cost is *not* raw $\text{KL}(q\|q')$ but the Eq. 11

margin-violation cost — a two-term ReLU penalty (acceptance-threshold + residual-replacement, at (α, β)) that is the quantity actually governing the $q' \to q$ gate's rejection rate. The paper shows DIMR-selected masks beat random/ evenly-spaced ones; our faithful search confirms this (∼50% lower cost than evenly-spaced).

3 Methodology (our extension)

Pipeline. Drafter p (0.5B) proposes a γ=5-token block; for each token a gate chooses one of three actions: *accept* p's token (cheap), *regenerate* with the slim verifier q' (a layer-skipped copy of the full verifier q=14B, medium), or *escalate* to q (expensive). The block commits up to the first changed token, then redrafts.

Learned gate. The fixed (α_1, α_2) thresholds are replaced by a tiny MLP over *q-free* features (drafter/q' entropy, top-1 prob, confidence ratio $q'(v)/\max q'$, p/q' agreement, KL$(p\|q')$, block position). Crucially, q is *never* a policy input — it is used only at training time to compute rewards/labels — so the gate is deployable without running the full model to decide.

Slim verifier q' / DIMR. q' skips ∼45% of q's decoder layers (residual blocks ⇒ identity, dimensionally safe). The skip mask is found by an offline DIMR search minimizing the paper's Eq. 11 *margin-violation* cost between q and q' (the two-term ReLU acceptance/residual cost, not raw KL); the searched mask is ∼50% lower-cost than an evenly-spaced one.

Metrics. *q-calls/token* (full-verifier invocations per token; exact, hardware-independent), GSM8K accuracy, and speedup. SD's win is latency, but measured wall-clock is dominated by a ∼10 ms eager per-call overhead floor; we report an **overhead-free bandwidth-model speedup** (per-forward cost = active-params×2 bytes/HBM-bw), cross-validated by a measure-and-subtract estimate (they agree for single-tier methods).

4 Reinforcement Learning for the Gating Policy

4.1 Problem framing

Each drafted-and-gated token is a decision step: state s_t = the q-free feature vector, action $a_t \in \{\text{ACCEPT}, \text{REGEN}, \text{ESCALATE}\}$, policy π_θ a 2-layer MLP. Routing effects are largely *local* (the action at t sets token t and perturbs nearby context), which motivates a contextual-bandit view. All policies are **warm-started from an imitation policy** (below); RL then refines on the policy's *own* rollout distribution.

4.2 Imitation warm-start and exposure bias

An oracle that may consult q labels the cost-minimal route reproducing greedy-q (ACCEPT if p's token = $\arg\max q$; REGEN if $\arg\max q' = \arg\max q$; else ESCALATE); the MLP is fit with class-weighted cross-entropy. Imitation alone reaches only ∼0.33 GSM8K accuracy: trained on oracle-visited (near-greedy) states, it over-accepts at its *own* states where errors compound — classic **exposure bias**. RL's primary value here is fixing this by training on-policy, lifting accuracy to 0.88+.

4.3 Reward design

The decoding goal is to reproduce the target model's output cheaply, giving two ingredients: *quality* and *cost*.

- **Quality (dense proxy).** $\text{match}_t = \mathbf{1}[\,\text{emitted}_t = \arg\max q(\cdot \mid x_{<t})\,]$, computed by running q at every position *at training time only* (not charged to cost).
- **Cost.** Per-action latency in *overhead-corrected* units: $\text{cost}_t = (t_{p1} + t_{q'}/\gamma + \mathbf{1}[\textsc{escalate}]\, t_q)/t_{q1}$ — accept ≈ 0.1, escalate ≈ 1.5.
- **Correctness (sparse, true objective).** correct $\in \{0, 1\}$, GSM8K final-answer match.

We compare three reward forms:

$$\text{REINFORCE/GRPO (trajectory):} \quad R = \text{match\_rate} + r_c\,\text{correct} - \lambda\,\overline{\text{cost}} \tag{1}$$

$$\text{v2 (per-token):} \quad r_t = \text{match}_t - \lambda\,\text{cost}_t + \tfrac{r_c\,\text{correct}}{T} \tag{2}$$

$$\text{correctness-focused GRPO:} \quad R = \text{correct} - \lambda\,\overline{\text{cost}} \tag{3}$$

The proxy gap. `match` is a *dense surrogate* for the *sparse* true objective (correctness). We measured the disconnect: a policy matching greedy-q on 98.5% of tokens still scores only $\sim$0.72 GSM8K accuracy, because a few deviations per chain derail the final answer. This motivates the correctness-focused reward (Eq. 3), which optimizes the true objective directly; its ceiling is bounded by the q-free features (a confidently-*wrong* drafter token looks safe without q). A cost penalty is retained or it collapses to "escalate everything" = greedy.

4.4 Credit assignment: per-trajectory vs. per-token

Trajectory-level (REINFORCE, GRPO). One scalar R per rollout is applied to *all* $\sim$150 routing decisions: $\nabla_\theta J = \mathbb{E}[A \sum_t \nabla \log \pi(a_t \mid s_t)]$. Crude — a good and a bad decision in the same rollout receive identical credit — and high-variance. **Per-token (v2).** Each decision is treated as a local bandit step with its *own* r_t and advantage, exploiting the locality of routing. This is far lower-variance and more precise; empirically it produces the smoothest training curves (Fig. 3) and the lowest cost/token.

4.5 Advantage estimation: REINFORCE vs. GRPO

REINFORCE uses $A = R - b$ with b a *global EMA* of reward across *all* prompts. This conflates per-*problem* difficulty (easy prompts give high R regardless of routing) with policy quality, injecting variance. **GRPO** samples a *group* of G rollouts of the *same* prompt and sets $A_i = (R_i - \text{mean}_{\text{group}})/(\text{std}_{\text{group}} + \epsilon)$, cancelling problem difficulty with no value network; it adds a PPO-clipped surrogate (clip 0.2, K=3 inner epochs), a KL penalty to the frozen warm-start reference, and an entropy bonus. GRPO is the right tool for the *sparse correctness* reward (Eq. 3): the group baseline yields a gradient precisely on the mixed-difficulty prompts where some of the G rollouts succeed and others fail. **v2** uses batch-normalized per-token advantages (contextual-bandit REINFORCE) with an entropy bonus and gradient clipping.

4.6 Gradient updates

Let $\pi_\theta(a_t \mid s_t)$ be the gate and $\mathcal{H}$ its entropy. The three methods maximize $J(\theta)$ by the following gradients (all with an entropy bonus $+\eta\,\mathcal{H}[\pi_\theta]$):

$$\text{REINFORCE:}\quad \nabla_\theta J = \mathbb{E}\Big[(R-b)\textstyle\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big], \qquad b \leftarrow (1-\tau)\,b + \tau\,R \tag{4}$$

$$\text{GRPO:}\quad \nabla_\theta J = \mathbb{E}\Big[\textstyle\sum_t \nabla_\theta \min\big(\rho_t A_i,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,A_i\big)\Big] - \beta\,\nabla_\theta \mathrm{KL}\big[\pi_\theta \| \pi_{\text{ref}}\big],$$

$$\rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \qquad A_i = \frac{R_i - \mathrm{mean}_{j\in\mathcal{G}} R_j}{\mathrm{std}_{j\in\mathcal{G}} R_j + \varepsilon} \tag{5}$$

$$\text{per-token (v2):}\quad \nabla_\theta J = \mathbb{E}\Big[\textstyle\sum_t \hat{A}_t\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big], \qquad \hat{A}_t = \frac{r_t - \mathrm{mean}_{\text{batch}}\, r}{\mathrm{std}_{\text{batch}}\, r + \varepsilon} \tag{6}$$

where R is the trajectory return (Eq. 1/3), r_t the per-token reward (Eq. 2), b a global EMA baseline (decay τ), $\mathcal{G}$ a group of G rollouts of one prompt, π_{ref} the frozen warm-start policy, and (ϵ, β, η) the clip / KL / entropy coefficients. REINFORCE and GRPO broadcast one (trajectory) advantage over all steps; v2 assigns each step its own $\hat{A}_t$.

4.7 What we observed

Fig. 3: **v2 per-token** is the lowest-variance and drives rejection lowest; **GRPO** is noisiest (group-level binary-ish reward); **REINFORCE** is intermediate; all converge to match$\approx$0.95. On the efficiency metric, GRPO pushed q-calls/token lowest among the match-rewarded variants (0.162), while the v2 reward drove escalation to 0.12 (fastest). Because the match-based rewards are match-dominated, the resulting RL is close to *on-policy imitation* (it fixes exposure bias rather than discovering exotic routing). Optimizing the *true* sparse objective (Eq. 3) directly is harder. We swept λ and the warm-start base (v2, REINFORCE+DIMR, REINFORCE-naive); *every* correctness/accuracy-floor variant we tried **degraded its warm-start** (final accuracy 0.46/0.60/0.82/0.82, all below the respective base) — a cost-dominated drift toward more acceptance or mis-targeted escalation that the q-free features cannot correct. The match-proxy-trained `REINFORCE+DIMR` policy (0.907) remains our best accuracy operating point; directly optimizing correctness does not beat it at this scale.

5 Results

Method	GSM8K acc	q-calls/tok	speedup
greedy-q (reference)	0.887	1.000	1.00×
plain SD (standard, lossless)	0.907	0.259	1.40×†
via_fixed (paper, tuned)	0.40*	0.31	1.76×
via_imit (imitation only)	0.34	0.29	1.69×
via_rl REINFORCE	0.880	0.243	2.29×
via_rl GRPO	0.853	**0.162**	2.94×
via_rl REINFORCE + DIMR	**0.907**	0.250	2.22×
via_rl v2 per-token (λ=0.3)	0.713	0.104	**3.53**×

Table 1: Headline comparison (0.5B→14B, GSM8K). †plain SD speedup is **measured wall-clock** (Qwen) at 0.907 accuracy — the realistic SD anchor (matches the paper's 1.3–1.7×). All other speedups are the **bandwidth model** (idealized, overhead-free; the via_* real wall-clock would be lower), so they are not directly comparable to †. *via_fixed peaks at ≈0.50 even under a full 9-config threshold sweep. q-calls/token is exact and hardware-independent. Accuracy n=80–150.

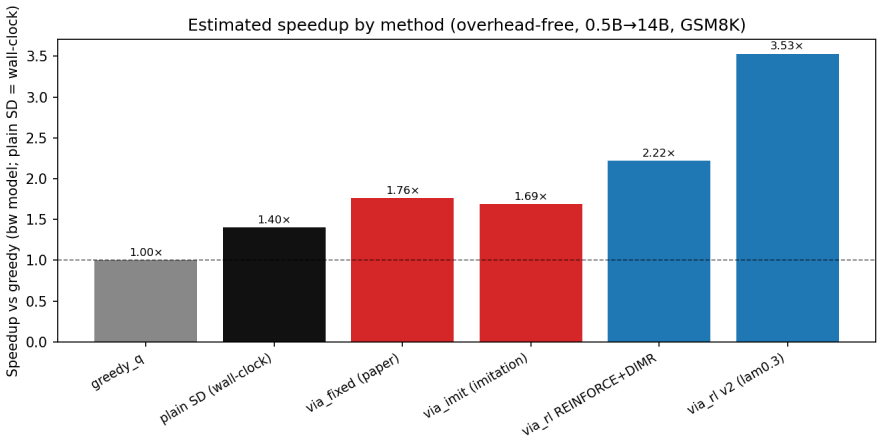

Figure 2: Speedup by method. **plain SD's bar is measured wall-clock (1.4×, Qwen)**; all via_* bars are the idealized bandwidth model (overhead-free) and are *not* directly comparable — their real wall-clock would be lower. The fixed-threshold gate over-escalates.

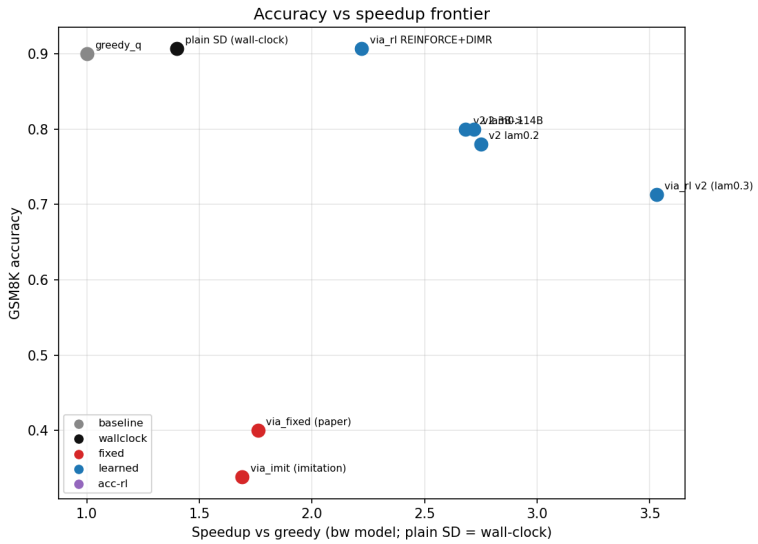

Figure 3: Accuracy vs. speedup. Learned gates dominate fixed ones; `REINFORCE+DIMR` reaches near-greedy accuracy (0.907); v2 trades accuracy for higher speed. (plain SD's x is measured wall-clock; via_* use the bandwidth model — different metrics.)

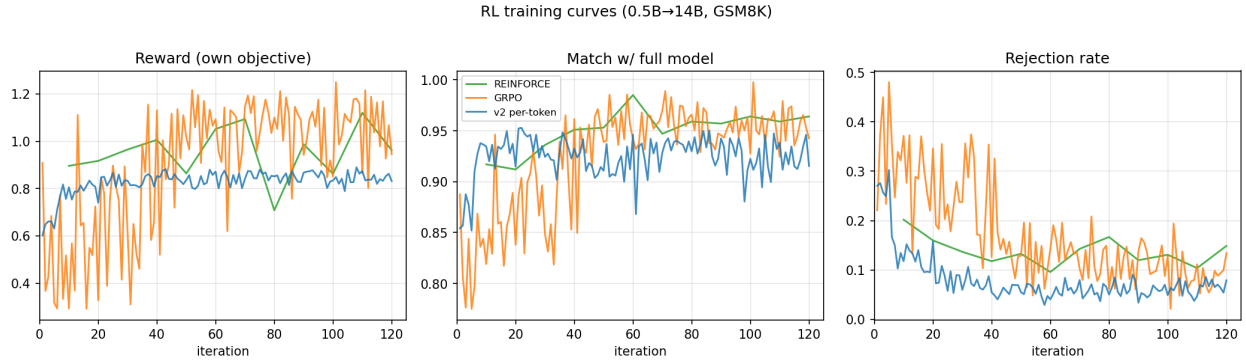

Figure 4: RL training curves. v2 per-token (blue) is lowest-variance and drives rejection lowest; GRPO (orange) is noisiest; REINFORCE (green) intermediate; all reach match≈0.95.

6 Findings

1. **Learned gate ≫ fixed thresholds** (0.88–0.91 vs. 0.21–0.40, and ≤ 0.50 fully tuned), at lower q-calls/token — robust across masks and capacity gaps. *Mechanism:* the layer-skipped q' is miscalibrated, so a single global confidence cutoff cannot separate confident-and-correct from confident-but-wrong tokens; the fixed rule accepts locally-plausible but globally-wrong tokens, and on GSM8K (all-or-nothing final answer) those per-token errors compound — which is why even a full (α_1, α_2) sweep plateaus at ≤ 0.50.
2. **RL fixes imitation's exposure bias** (0.33 → 0.88+) by training on-policy.
3. **DIMR helps only the learned gate** (via_rl 0.88 → 0.91); the fixed rule stays brittle.

4. **Credit assignment matters**: per-token (v2) is far lower-variance than trajectory-level REINFORCE/GRPO and reaches the lowest escalation; GRPO's group baseline beats REINFORCE's global one.
5. **The v2 per-token reward drives escalation below break-even** and reaches the highest speedup (3.53×) but overshoots accuracy (0.71) at λ=0.3; lower λ trades back toward accuracy (λ=0.1: acc 0.80 at 2.68×).
6. **Honest negatives**: the q' middle tier's per-block overhead limits net speedup at this gap; and a bigger *verifier* (32B) hurts an accept-heavy policy (acc 0.33) because the fixed small drafter is then relatively weaker — a bigger *drafter* is the indicated lever.

7 Caveats

Accuracy at small n (24–50) carries a ±0.06–0.10 CI, so small accuracy gaps are within noise (large ones are not). Speedups are an overhead-free cost *model*, not a stopwatch on an optimized engine (CUDA-graphs/static-cache were incompatible with the dynamic KV-cache loop). Single task (GSM8K), single model family (Qwen2.5).

Why our accuracy differs from the paper. Our plain-SD GSM8K accuracy (0.907) is well above the paper's reported SD numbers (0.70 for Gemma2-2B→9B, 0.78 for 2B→27B). This reflects the *base model, not the method*: standard SD is *lossless*, so its accuracy equals the target verifier's own greedy accuracy, and Qwen2.5-14B is far stronger on GSM8K than Gemma2-9B/27B. The methodologically meaningful agreement is that, in both works, SD accuracy tracks the verifier's greedy accuracy and the hierarchical/learned variants preserve or marginally shift it. Within our own runs the SD (=greedy) accuracy also varies by ±0.02 from small n plus bf16 nondeterminism between the cached and re-forward code paths (which is also why our measured plain_sd $\approx 0.907 >$ greedy$_q \approx 0.887$ despite SD being exactly lossless). We also use 4-shot prompting vs. the paper's 8-shot.